

Pemrograman Berorientasi Objek

Kode MK : CAK2KAB4

Dany Candra Febrianto, S.Kom., M.Eng.

Pengampu

- Nama : Dany Candra Febrianto, S.Kom., M.Eng.
- Home Base : Prodi Informatika Direktorat Kampus Purwokerto
- Kontak : 089667194026
- Email : danycandra@telkomuniversity.ac.id

Deskripsi Mata Kuliah

Mata kuliah ini mempelajari konsep-konsep pada *object oriented programming* (OOP), seperti *class* dan *object*, *encapsulation*, *inheritance*, *interface*, dan *abstract class*. Secara spesifik konsep-konsep tersebut akan dipelajari dengan bantuan Bahasa pemrograman Java. Selain konsep OOP, pada mata kuliah ini juga akan dipelajari *class diagram*, *exception*, dan JDBC sebagai dasar pembangunan aplikasi berorientasi objek. Selain itu akan dipelajari juga arsitektur *Model View Controller* (MVC) sebagai arsitektur standar dalam pembangunan aplikasi berorientasi objek.

Program Learning Outcome (PLO)

PLO 8 : Kemampuan mengimplementasikan bahasa pemrograman dan tools yang relevan untuk memberikan solusi di bidang informatika

Course Learning Program (CLO)

CLO 2 - Mampu mengimplementasikan bahasa pemrograman dan tools perangkat lunak yang relevan

Sub – CLO :

Sub-CLO-8-2-1	Mahasiswa mampu memahami konsep OO
Sub-CLO-8-2-2	Mahasiswa mampu mengimplementasikan OO untuk pengembangan aplikasi
Sub-CLO-8-2-3	Mahasiswa mampu membuat pemodelan sistem berbasis OO dengan menggunakan class diagram

Komposisi Penilaian

Id Sub-CLO	Bobot per Bentuk Penilaian (%)						TOTAL BOBOT PER Sub-CLO
	Aktivitas Partisipatif	Kognitif/Pengetahuan			Hasil Proyek		
	Keaktifan/Kehadiran	Kuis/Tugas	Praktikum	UTS	Tugas Besar	Tugas Dokumentasi	
Sub-CLO-8-2-1		8	12	20			40
Sub-CLO-8-2-2	5	6	9		30		50
Sub-CLO-8-2-3		1	2	2		5	10
Total per penilaian	5	15	23	22	30	5	100

Assesmen, Kuis, Praktuikum dan Tugas

- Assesmen Tertulis (UTS)
- Kuis dikerjakan melalui LMS
- Praktikum berupa laporan Praktikum
- Tugas (Tugas Dokumentasi dan Tugas Besar)

Referensi

- Head First Java. Kathy Sierra, Bert Bates, Trisha Gee. 3rd ed. 2022, O'Reilly [UTAMA]
- Java How to Program. Paul J. Deitel & Harvey Deitel. 11th ed. 2017, Pearson [UTAMA]
- The Java Language Specification. James Gosling et. al. Java SE 11 Edition. 2018, Oracle America Inc [PENDUKUNG]
- Logika Pemrograman Java. Abdul Kadir. 2023, Elex Media Komputindo [PENDUKUNG]

Peraturan Kelas

- Kehadiran minimal 75%.
- Terlambat Maksimal 15 menit dalam perkuliahan.
- Tidak ada susulan untuk Kuis, Laporan Praktikum dan Tugas.
- Susulan ASESMEN (UTS) hanya diberikan dikarenakan mahasiswa sakit atau dispensasi kampus.
- Kecurangan berupa plagiarisme (khususnya tugas, laporan atau project dsb) akan menyebabkan nilai MK menjadi E.

Silabus

- Week 1 : Intro OOP, Class and Object
- Week 2 : Encapsulation
- Week 3 : Class Relationship
- Week 4 : Inheritance
- Week 5 : Abstract Class and Interface
- Week 6 : Polymorphism, Static, & Collection
- Week 7 : Exception
- Week 8 : Mid-term Exam (written test)
- Week 9 : MVC 1: Web Frontend
- Week 10 : MVC 2: Java Web
- Week 11-12 : JDBC
- Week 13 : Session
- Week 14 - 15: Tubes Presentation
- Week 16 : Review dan Remed

Agenda

- Konsep Pemrograman Berorientasi Objek
- Kelas dan Objek
- *Attribute, Behaviour (Procedure & Function)*
- *Visibility*
- *Constructor*

Kenapa Harus Belajar OOP ?

OOP dikenal sebagai teknik pemrograman modern yang banyak digunakan pada *Framework di* Banyak Bahasa Pemrograman.

Apa itu OOP?

- OOP (*Object Oriented Programming*) atau dalam bahasa indonesia dikenal dengan pemrograman berorientasikan objek (PBO) merupakan sebuah paradigma atau teknik pemrograman yang **berorientasikan Objek**.
- Pada OOP, Fungsi dan variabel **dibungkus** dalam sebuah **objek** atau *class* yang dapat saling berinteraksi, sehingga membentuk sebuah program. Sehingga tidak ada lagi kode yang berantakan karena redundansi.

Tujuan OOP

Dengan memahami dan menerapkan konsep-konsep ini, kita dapat menulis kode yang lebih **modular, mudah dipelihara, dan dapat diulang**, yang pada akhirnya menghasilkan pengembangan perangkat lunak yang lebih efisien dan berkualitas tinggi.

4 Pilar OOP



ENCAPSULATION



ABSTRACTION



INHERITANCE



POLYMORPHISM



(Dive Into Design Patterns, Alexander Shvet 2019)

Enkapsulasi

Enkapsulasi adalah konsep yang menyatukan data (atribut) dan perilaku (metode) yang beroperasi pada data tersebut ke dalam satu unit yang disebut objek, serta **melindungi data dari akses langsung dari luar objek.** Enkapsulasi dicapai dengan membatasi akses ke atribut dan metode melalui penggunaan modifier akses seperti private, protected, dan public.

(Dive Into Design Patterns, Alexander Shvet 2019)

Abstraksi

Abstraksi adalah konsep yang memungkinkan kita untuk menyederhanakan kompleksitas dengan hanya menampilkan atribut dan perilaku yang relevan dari suatu objek, sambil menyembunyikan detail implementasinya. Dalam OOP, abstraksi dicapai melalui penggunaan kelas dan objek. Kelas mendefinisikan karakteristik umum dari suatu objek, sementara objek adalah instansi dari kelas tersebut yang memiliki nilai spesifik.

(Dive Into Design Patterns, Alexander Shvet 2019)

Pewarisan

Pewarisan (inheritance) adalah mekanisme dalam OOP di mana sebuah kelas dapat mewarisi atribut dan metode dari kelas lain. Ini memungkinkan kita untuk membuat kelas baru yang disebut "subclass" atau "anak" berdasarkan kelas yang sudah ada yang disebut "superclass" atau "induk". Pewarisan mempromosikan penggunaan kembali kode dan mempermudah pengelompokan objek yang memiliki karakteristik serupa.

(Dive Into Design Patterns, Alexander Shvet 2019)

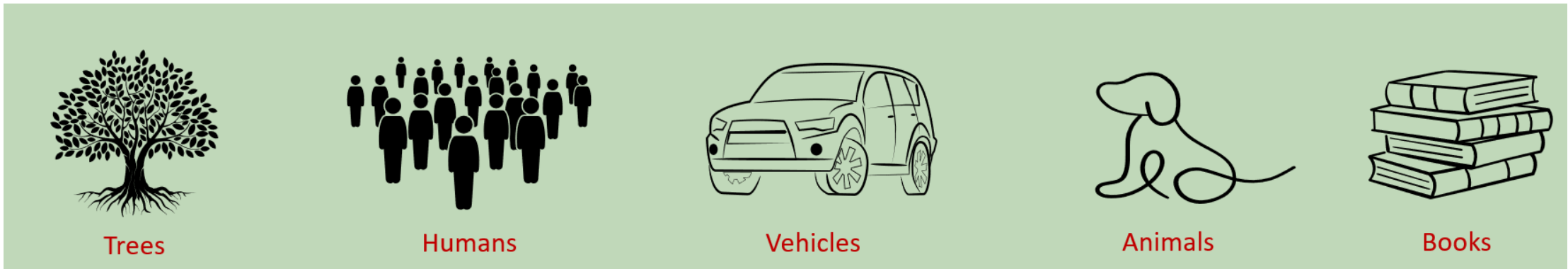
Polimorfisme

Polimorfisme adalah konsep di mana sebuah metode atau fungsi dapat memiliki bentuk yang berbeda berdasarkan kelas yang memanggilnya. Dengan kata lain, **metode yang sama dapat berperilaku berbeda tergantung pada objek mana yang memanggilnya.** Polimorfisme memungkinkan kita untuk memprogram secara lebih fleksibel dan dinamis.

(Dive Into Design Patterns, Alexander Shvet 2019)

Apa itu Objek?

Analogi Objek adalah semua hal yang kita bisa lihat di kehidupan nyata



Setiap objek yang kamu lihat memiliki **Attribute** dan **Behaviour**.
Sebagai contoh, seorang manusia dapat memiliki

Attribute : nama, jenis kelamin, usia, tinggi badan, berat badan,

Behaviour : berjalan, berbicara, berlari, belajar, bekerja, tidur, dan sebagainya.

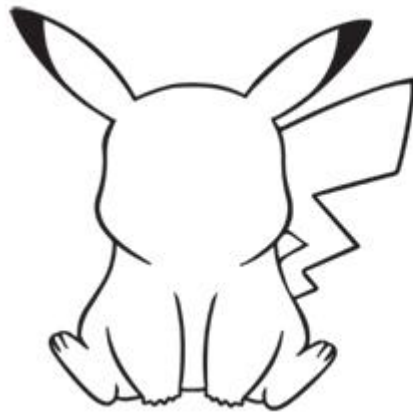
Apakah atribut-atribut tersebut
akan sama untuk semua
manusia?

Class vs Objek

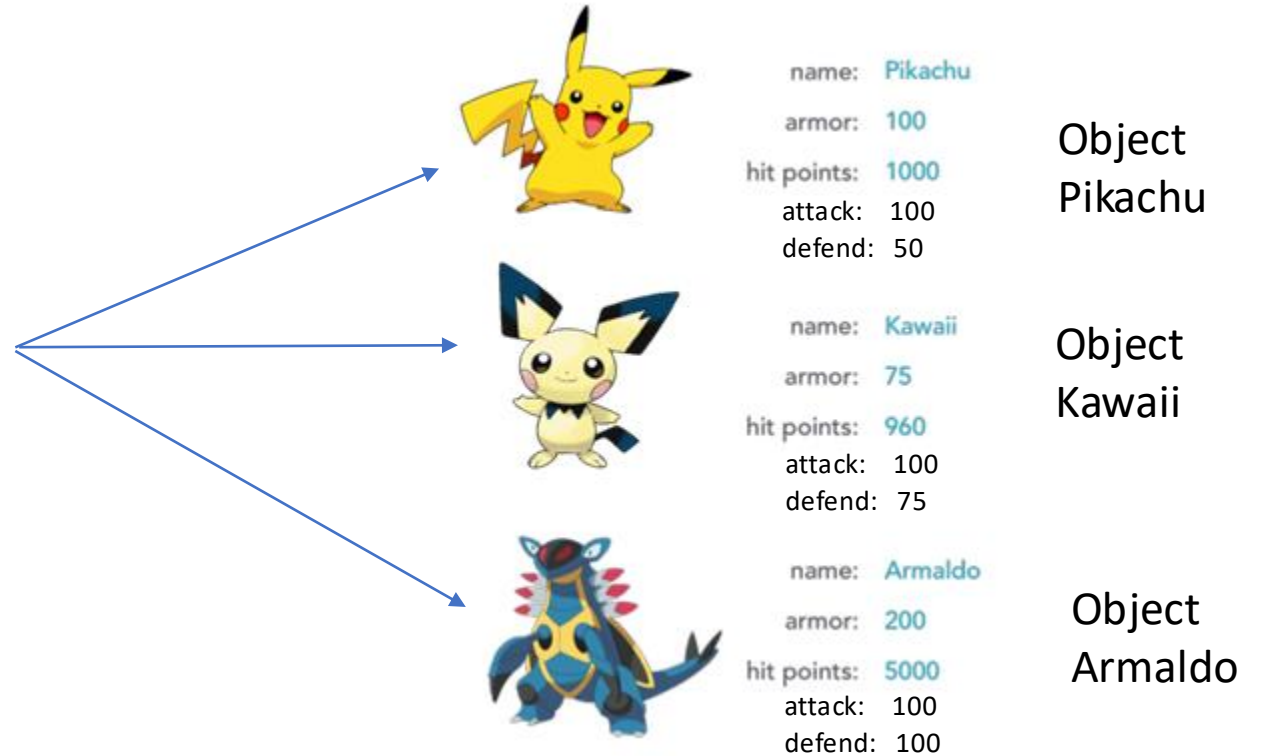
- *Class adalah rancangan atau blue print dari sebuah objek.*
- Objek adalah sebuah variabel yang merupakan *instance* atau perwujudan dari *Class*.

Analogi Class

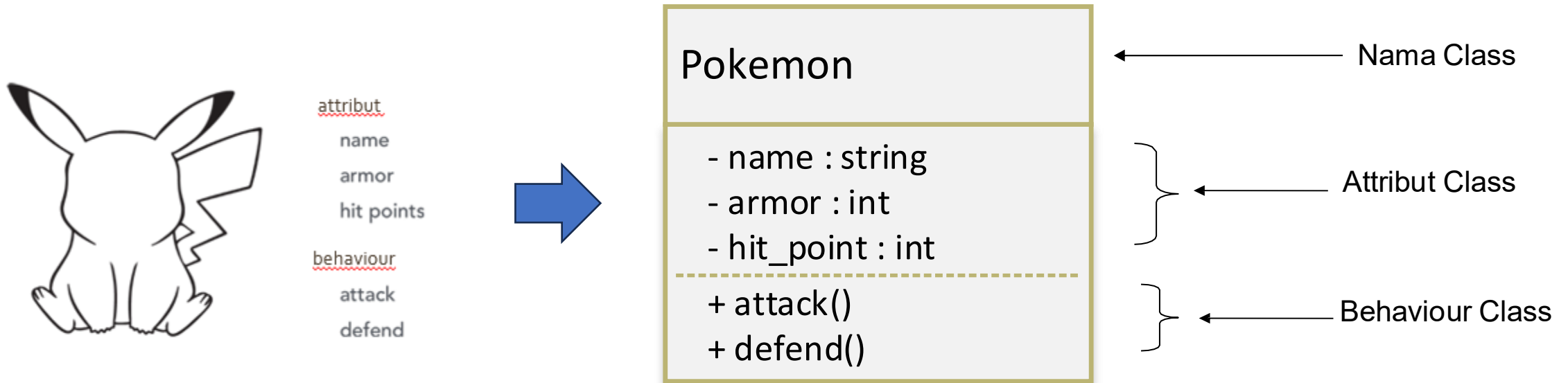
Class Pokemon



attribut
name
armor
hit points
behaviour
attack
defend



Komponen Class



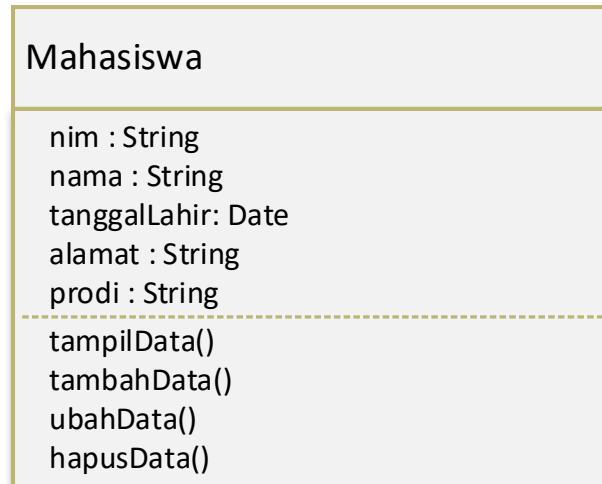
Deskripsi :

Nama Class : Sesuatu untuk menjadi (identifier) penamaan dari sebuah kelas

Attribut Class : Sesuatu yang menjadi ciri dari sebuah kelas

Behaviour Class : Sesuatu yang dapat dilakukan oleh kelas

Contoh :



```
public class Mahasiswa {  
    // Atribut  
    private String nim;  
    private String nama;  
    private Date tanggalLahir;  
    private String alamat;  
    private String prodi;  
  
    public void tampilData() {  
        // Method untuk menampilkan data mahasiswa  
    }  
  
    public void tambahData() {  
        // Method untuk menambahkan data mahasiswa  
    }  
  
    public void ubahData() {  
        // Method untuk mengubah data mahasiswa  
    }  
  
    protected void hapusData() {  
        // Method untuk menghapus data mahasiswa  
    }  
  
}
```


Kelas

- Kelas merupakan kerangka kode yang berisikan struktur atribut, struktur prosedur dan struktur fungsi yang digunakan untuk membuat objek.
- Sebuah kelas dapat menghasilkan lebih dari satu objek.

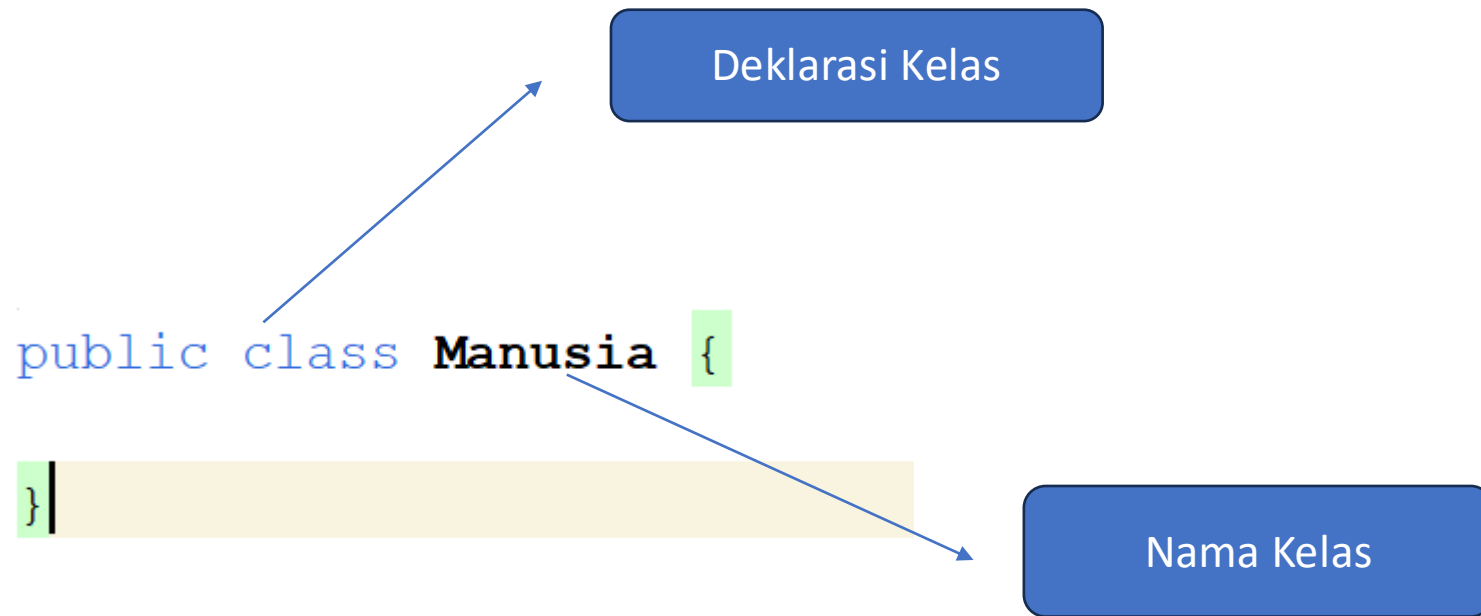
Membuat Kelas

- Nama kelas harus sama dengan nama File.
- Nama kelas **tidak boleh mengandung huruf unik** (@, #, \$, %, &, dll) **dan huruf *whitespace*** (spasi, enter, tab, dll)

```
class NamaKelas {
```

```
    // isi kelas
```

```
}
```



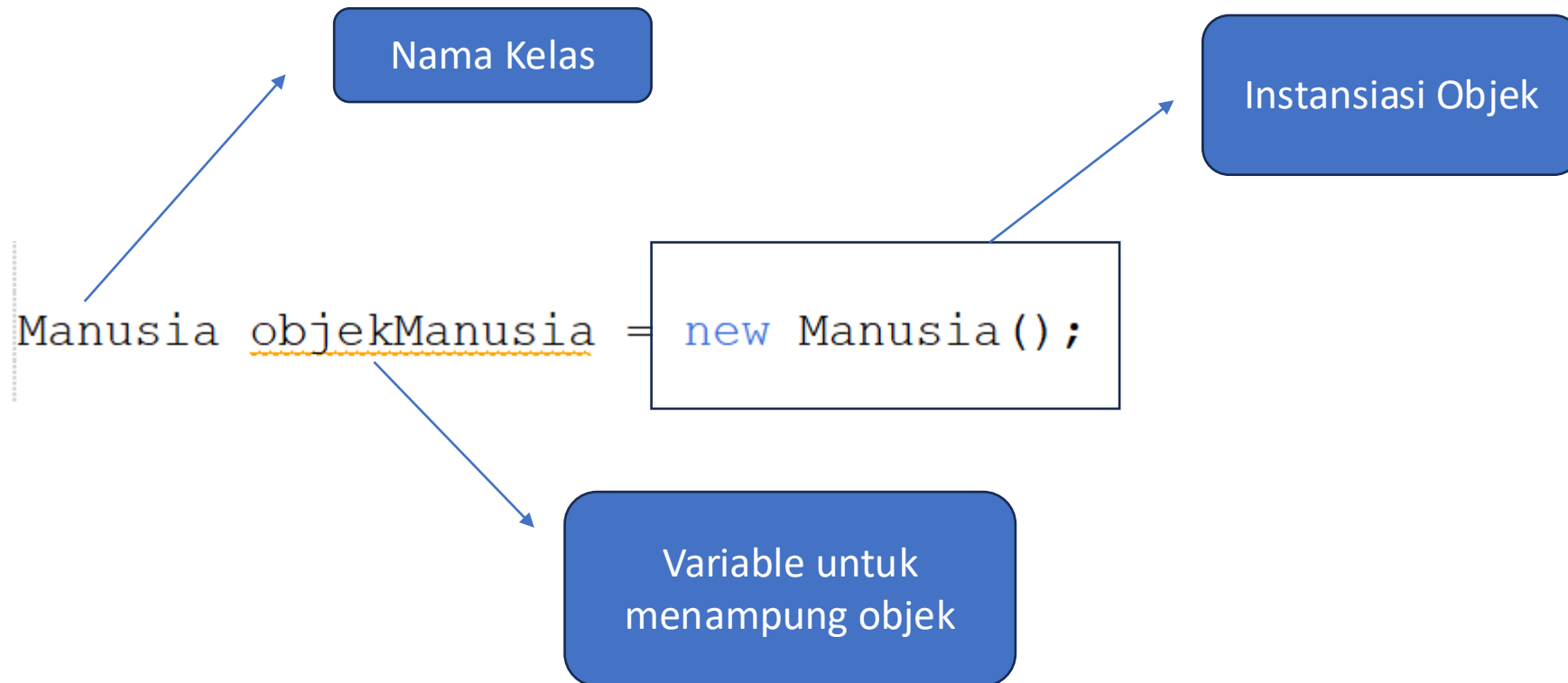
Object

- Objek merupakan hasil instansiasi dari kelas.
- Untuk membuat objek, di Java dapat menggunakan perintah **new**.
- Atribut, metode dan fungsi yang dimiliki oleh objek, hanya yang dideklarasikan dalam kelas.

Membuat Object

- Objek merupakan variabel yang memiliki tipe data Kelas
- Kriteria pembuatan objek sama seperti pembuatan variabel.

NamaKelas object = new NamaKelas();



Atribut

- Atribut merupakan data atau sifat yang dimiliki oleh sebuah kelas.
- Atribut dibuat layaknya sebuah variabel.

Membuat Atribut

```
class NamaKelas {
```

```
    TipeData namaAtribut;
```

```
}
```


Manipulasi Atribut

- Atribut dapat diakses lewat kelas.
- Untuk mengakses atribut, dapat menggunakan tanda . (titik)

```
NamaKelas objek = new NamaKelas();  
objek.namaAtribut = ...;
```

```
public class Orang {
```

```
    String nama;  
    String alamat;  
    int umur;  
    boolean menikah;
```

```
}
```

Atribut

Cara memberi nilai pada atribut

```
Orang budi = new Orang();  
budi.nama = "Budi";  
budi.alamat = "Bandung";  
budi.umur = 25;  
budi.menikah = false;
```

Prosedur

- Prosedur merupakan sebuah kode program yang dapat digunakan untuk menjalankan instruksi program dalam sebuah kelas.
- Sebuah kelas dapat memiliki lebih dari satu prosedur.
- Nama prosedur tidak boleh sama dengan prosedur lain.

Membuat Prosedur

```
class NamaKelas {  
  
    void namaProsedur(){  
        // isi prosedur  
    }  
  
}
```

Menggunakan Prosedur

```
NamaKelas objek = new NamaKelas();  
objek.namaProsedur();
```

```
public class Manusia {
```

```
    String nama;
```

```
    String jenisKelamin;
```

```
    String alamat;
```

Cara menuliskan prosedur didalam kelas

```
    void tampilInformasi() {
```

```
        System.out.println("Nama : " + nama);
```

```
        System.out.println("Jenis Kelamin : " + jenisKelamin);
```

```
        System.out.println("Alamat : " + alamat);
```

```
    }
```

```
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Manusia budi = new Manusia();  
    budi.nama = "Budi Nugraha";  
    budi.jenisKelamin = "Laki-Laki";  
    budi.alamat = "Cirebon";  
    budi.tampilInformasi();  
}
```

cara memaminggil prosedur

Fungsi

- Fungsi merupakan kode program yang digunakan untuk menghasilkan sesuatu.
- Untuk mengembalikan hasil, dapat menggunakan perintah **return** diikuti dengan data yang dihasilkan.
- Fungsi hanya dapat menghasilkan sebuah nilai.
- Sebuah kelas dapat memiliki lebih dari satu fungsi.
- Nama fungsi tidak boleh sama dengan fungsi yang lain.

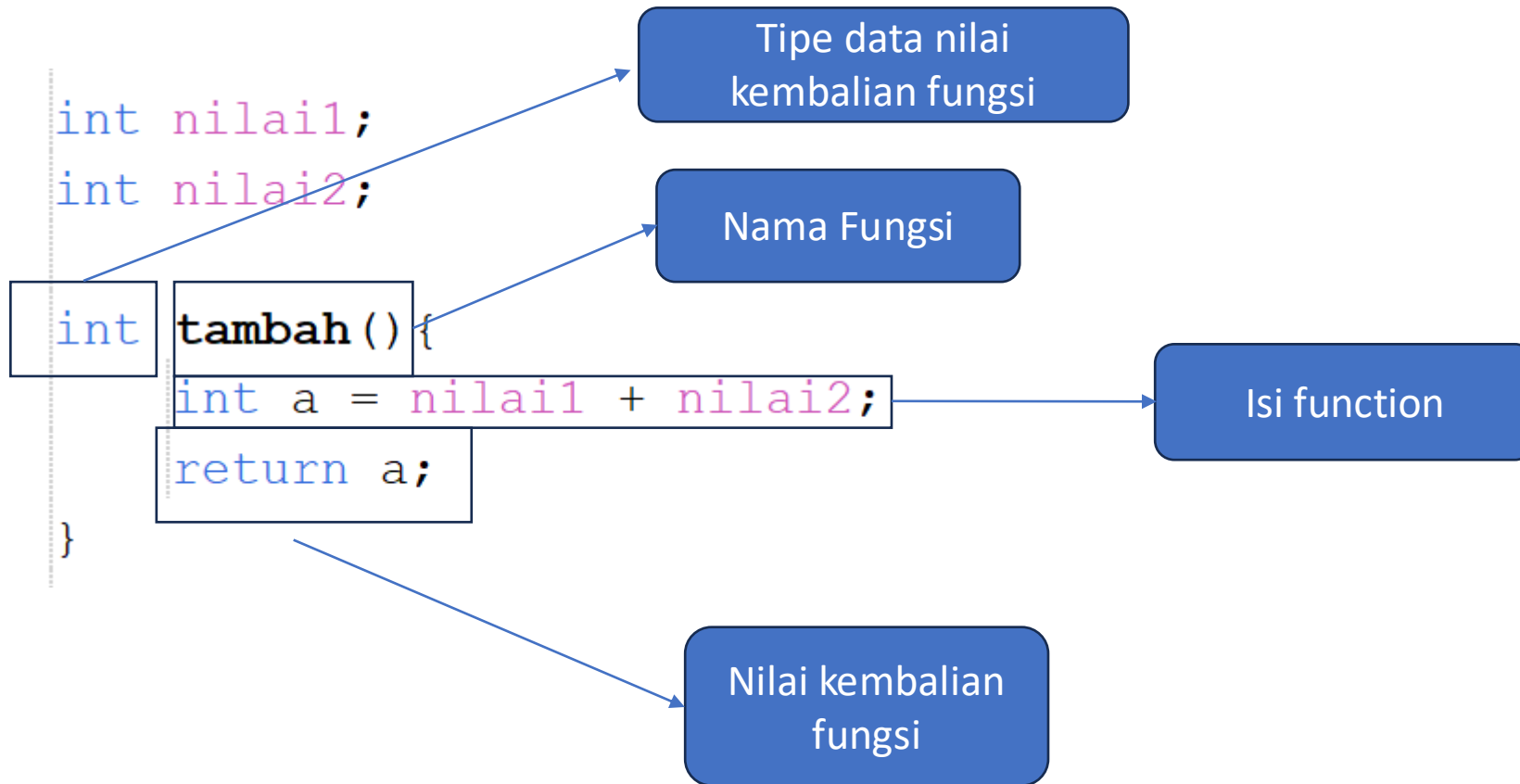
Membuat Fungsi

```
class NamaKelas {  
  
    TipeData namaFungsi(){  
        // kode program  
        return hasil;  
    }  
  
}
```

Menggunakan Fungsi

NamaKelas objek = new NamaKelas();

TipeData hasil = objek.namaFungsi();



Parameter

- Parameter merupakan data yang dapat disisipkan ke dalam prosedur dan fungsi.
- Jumlah parameter dalam prosedur dan fungsi dapat lebih dari satu.

Membuat Parameter

```
class NamaKelas {  
    void NamaProsedur(TipeData parameter){  
        // kode program  
    }  
    TipeData namaFungsi(TipeData param1, TipeData param2){  
        // kode program  
        return hasil;  
    }  
}
```

Menggunakan Parameter

```
NamaKelas objek = new NamaKelas();  
objek.namaProsedur("String");
```

```
TipeData hasil = objek.namaFungsi(100, 300);  
TipeData variabel = ...;  
objek.namaProsedur2(variabel);
```

```
int tambah(int nilai1, int nilai2){  
    int hasil = nilai1 + nilai2;  
    return hasil;  
}
```

Parameter untuk masukan
dalam fungsi atau prosedur

Visibility

- Visibility merupakan kemampuan untuk mengontrol sebuah akses ke **kelas, atribut, prosedur** dan **fungsi**.
- Java mendukung beberapa visibility, yaitu :
 - Default
 - Public
 - Private, dan
 - Protected

Hak Akses Visibility

Visibility	Kelas	Paket	Sub Kelas	Umum
Private	Y			
Default	Y	Y		
Protected	Y	Y	Y	
Public	Y	Y	Y	Y

Membuat Visibility Public

```
public class NamaKelas {  
  
    public TipeData namaAtribut;  
  
    public void namaProsedur(){  
    }  
  
    public TipeData namaFungsi(){  
        return hasil;  
    }  
  
}
```

Membuat Visibility Private

```
public class NamaKelas {  
  
    private TipeData namaAtribut;  
  
    private void namaProsedur(){  
    }  
  
    private TipeData namaFungsi(){  
        return hasil;  
    }  
  
}
```

Konstruktor

- Konstruktor merupakan prosedur yang diakses secara langsung ketika membuat sebuah objek dibuat.
- Konstruktor dapat memiliki parameter dan juga dapat di *overloading*.
- Nama konstruktor harus sama dengan nama kelas.

Membuat Konstruktor

```
class NamaKelas {  
  
    NamaKelas(){  
    }  
  
    // overloading konstruktor  
    NamaKelas(TipeData parameter){  
    }  
  
}
```

Menggunakan Konstruktor

NamaKelas objek1 = new NamaKelas();

NamaKelas objek2 = new NamaKelas("parameter");

```
public class Binatang {
```

```
String nama;  
String jenis;  
int kaki;  
int tangan;
```

```
Binatang(String nama, String jenis){  
    System.out.println("konstruktor 2 dipanggil");  
  
    this.nama = nama;  
    this.jenis = jenis;  
}
```

Nama Konstruktor

Parameter pada konstruktor
(boleh ada boleh tidak)

Isi dari konstruktor

Terima Kasih

Next : Encapsulation